

MICROPROCESSEUR MOTOROLA 6809

PROJET DE MODULE :

Architecture d'ordinateur

RÉALISER PAR :

OULED EL MOKADDEM YOUSRA.

HAMIM MAROUANE.

ENCADRÉE PAR :

Mr. HICHAM BENAALLA

Année Universitaire : 2025 – 2026

Table des matières

1. INTRODUCTION :	2
2. Architecture Générale du système	2
2.1 . Les étapes d'exécution	3
2.2. Le cycle d'exécution.	3
FETCH :	3
DECODE :	4
EXECUTE :	4
3. Analyse et Spécification	4
3.1. Description des Modules	6
3.1.2 . Module Mémoire	6
3.1.2. Module Registres :	7
3.1.3. Module Décodeur d'instructions :	8
3.1.4 Module Exécution	9
3.1.5. Interface Utilisateur (optionnel mais recommandé).	10
4. Réalisation (Développement).	11
4.1. Mémoire	11
4.2. CPU	13
4.3. Interface Graphique.	15
5. Travail en groupe.	16
6. Résultats & Tests.	17
7. Conclusion et Remerciements	19

RAPPORT

1. INTRODUCTION :

Ce projet consiste en un simulateur de microprocesseur Motorola 6809 développé en Java, qui reproduit fidèlement le fonctionnement interne et les mécanismes d'exécution de ce processeur emblématique. L'application dispose d'une interface graphique conviviale qui facilite la visualisation et la compréhension des opérations du processeur. Le simulateur émule les composants essentiels du 6809 et permet d'observer en temps réel l'évolution des registres, le contenu de la mémoire et l'exécution des instructions. Pour assurer une émulation fonctionnelle, j'ai implémenté trois modes d'adressage fondamentaux : le mode immédiat, qui charge directement les valeurs dans les instructions : le mode inhérent, qui opère uniquement sur les registres sans accès mémoire.

Le mode direct, qui accède aux données stockées en mémoire via des adresses courtes. Ce rapport détaille l'architecture logicielle adoptée, décrit le cheminement complet d'une instruction depuis sa saisie jusqu'à son exécution, et propose des scénarios d'utilisation pédagogique concrets pour faciliter la prise en main du simulateur.

2. Architecture Générale du système

Notre simulateur adopte une architecture modulaire ça veut dire que notre code est divisé en plusieurs modules chacun responsable d'une tâche , une

fonctionnalité précise dans l'émulation du Motorola 6809 pour simplifier le développement , la maintenance et l'extension de notre simulateur.

2.1 . Les étapes d'exécution

ETAPE 1 : Saisie des instructions L'utilisateur rédige son programme dans l'éditeur intégré de l'interface Cette interface offre une zone de texte EDITEUR permettant d'écrire des instructions en langage assembleur Motorola 6809.

ETAPE 2 : Les instructions validées sont converties en codes opératoires binaires correspondant au jeu d'instructions du 6809. Ces opcodes sont ensuite chargés dans la mémoire ROM pour l'exécution.

ETAPE 3 : L'unité de traitement lit séquentiellement les opcodes depuis la ROM, décode chaque instruction, et effectue les opérations requises en manipulant les registres et la mémoire RAM selon le mode d'adressage spécifié.

ETAPE 4 : Les modifications apportées aux registres et à la mémoire sont instantanément reflétées dans les interfaces graphiques correspondantes ce qui permet de suivre l'état du système en temps réel.

2.2. Le cycle d'exécution.

Dans le processeur, un algorithme interne travaille en arrière-plan : il gère les étapes de FETCH, DECODE et EXECUTE afin de préparer, interpréter et exécuter chaque instruction automatiquement. En d'autres termes, le programme ne fonctionne pas seul ; c'est le processeur qui organise et enchaîne ces opérations pour assurer le déroulement complet de l'exécution.

FETCH :

- Le PC envoie l'adresse de l'instruction sur le bus d'adresse.
- La mémoire renvoie l'octet de l'instruction vers le registre RI.

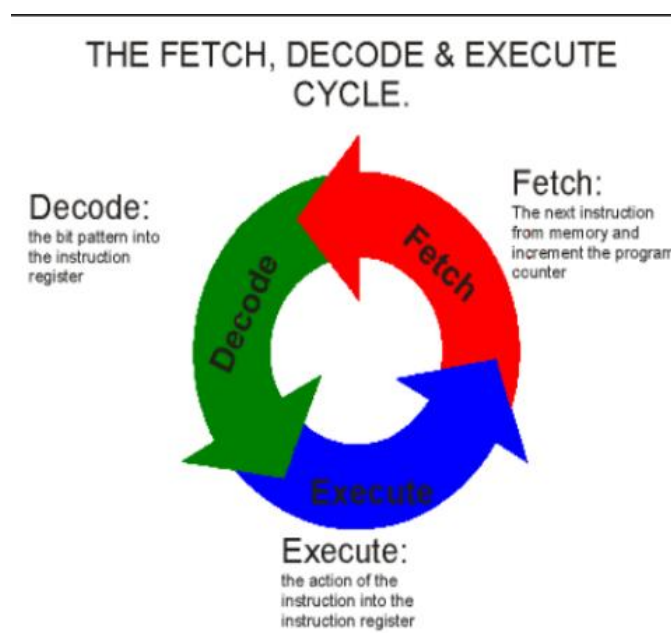
- Le PC s'incrémente pour pointer vers l'octet suivant.

DECODE :

- Le registre IR est analysé par l'unité de contrôle.
- L'opération, les registres utilisés et le mode d'adressage sont identifiés.
- Les signaux internes sont préparés (ALU, bus, accès mémoire, registres...).

EXECUTE :

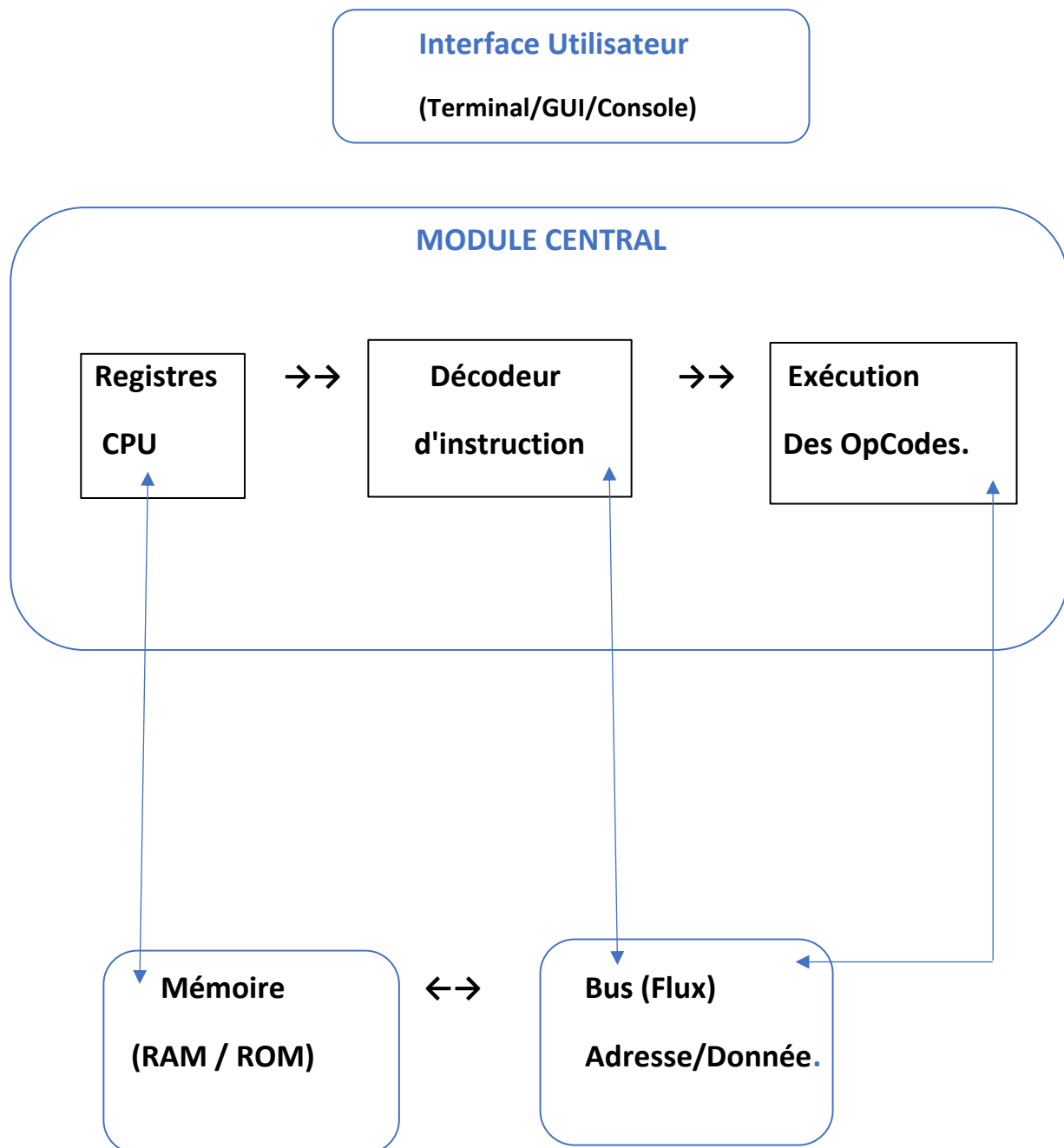
- L'opération est effectuée (calcul, transfert, accès mémoire, saut...).
- Les registres et les flags (N, Z, C, V) sont mis à jour selon le résultat.
- Le cycle revient au FETCH pour traiter l'instruction suivante.



3. Analyse et Spécification

Pour construire le simulateur du microprocesseur Motorola 6809, on adopte une architecture modulaire. Chaque partie du simulateur correspond à un

composant réel du processeur. Cela permet d'organiser le code, de faciliter les tests et de l'améliorer plus tard.



3.1. Description des Modules.

3.1.2 . Module Mémoire.

- Contient les adresses et les données comme la RAM d'un vrai CPU.
- Stocke le programme, les instructions et les variables.
- Permet : Lecture (Read) | Écriture (Write)
- Exemple : readByte(address) / writeByte(address, value).

```
public class Memory {
    private static final int MEMORY_SIZE = 65536;
    private RAM ram;
    private ROM rom;
    private boolean[] romMap;
    public Memory() {
        ram = new RAM();
        rom = new ROM();
        romMap = new boolean[MEMORY_SIZE];
        for (int i = 0; i < MEMORY_SIZE; i++) {
            romMap[i] = false;
        }
    }
}
```

```
public byte readByte(int address) {
    checkAddress(address);
    if (romMap[address]) {
        return rom.readByte(address);
    } else {
        return ram.readByte(address);
    }
}
```

```
public void writeByte(int address, byte value) {
    checkAddress(address);
    if (!romMap[address]) {
        ram.writeByte(address, value);
    } else {
        rom.writeByte(address, value);
    }
}
```

3.1.2. Module Registres :

Représente tous les registres internes du 6809 :

- **A, B** (accumulateurs)
- **D** (A+B combinés)
- **X, Y** (index)
- **SP** (stack pointer)
- **PC** (compteur ordinal)
- **CC** (registre d'état / flags)

Il sert à garder l'état du processeur à chaque étape.

```
package com.motorola6809.simulator;
import java.util.EnumSet;
public class CPU {
    private final Memory memory;
    private int regA;
    private int regB;
    private int regX;
    private int regY;
    private int regU;
    private int regS;
    private int regPC;
    private int regDP;
    private EnumSet<Flag> conditionCodes = EnumSet.noneOf(elementType: Flag.class);
    public CPU(Memory memory) {
        this.memory = memory;
    }

    public void initialize() {
        reset();
    }
    public int getRegA() {
        return regA;
    }
    public int getRegB() {
        return regB;
    }
    public int getRegPC() {
        return regPC;
    }
    public void reset() {
        regA = regB = 0;
        regX = regY = regU = regS = 0;
        regDP = 0;
        conditionCodes.clear();
        regPC = memory.readWord(address: 0xFFFFE);
    }
}
```

3.1.3. Module Décodeur d'instructions :

- Prend l'opcode venant de la mémoire
- Détermine quel type d'instruction exécuter
- Trouve le mode d'adressage utilisé (direct, immédiat, indexé...)
- Prépare les paramètres pour l'unité d'exécution

C'est comme un traducteur entre le programme binaire et les opérations internes.

```
public void step() {  
    int opcode = fetchByte();  
    switch (opcode) {  
        case 0x86:  
            opLDAimmediate();  
            break;  
        case 0xC6:  
            opLDBimmediate();  
            break;  
        case 0x8E:  
            opLDXimmediate();  
            break;  
        case 0xCE:  
            opLDUimmediate();  
            break;  
        case 0xCC:  
            opLDDimmediate();  
            break;  
    }
```

```
private void opLDAimmediate() {  
    regA = fetchByte();  
    updateNZFlags(regA);  
    clearVFlag();  
}  
private void opLDBimmediate() {  
    regB = fetchByte();  
    updateNZFlags(regB);  
    clearVFlag();  
}  
private void opLDXimmediate() {  
    regX = fetchWord();  
    updateNZFlags16(regX);  
    clearVFlag();  
}
```

3.1.4 Module Exécution

- Applique l'opération demandée (ADD, SUB, LDA, STA, JMP...)
- Met à jour les registres, les flags et la mémoire si nécessaire
- Retourne le contrôle au PC pour l'instruction suivante

C'est ici que "la magie du processeur" se produit.

```
private int getRegisterValue(int reg) {  
    switch (reg) {  
        case 0:  
            return regD() & 0xFFFF;  
        case 1:  
            return regX & 0xFFFF;  
        case 2:  
            return regY & 0xFFFF;  
        case 3:  
            return regU & 0xFFFF;  
        case 4:  
            return regS & 0xFFFF;  
        case 5:  
            return regPC & 0xFFFF;  
        case 8:  
            return regA & 0xFF;  
        case 9:  
            return regB & 0xFF;  
        case 10:  
            return getCC();  
        case 11:  
            return regDP & 0xFF;  
        default:  
            return 0;  
    }  
}
```

```

public void setPC(int address) {
    regPC = address & 0xFFFF;
}
public void setA(int value) {
    regA = value & 0xFF;
}
public void setB(int value) {
    regB = value & 0xFF;
}
public void setX(int value) {
    regX = value & 0xFFFF;
}
public void setY(int value) {
    regY = value & 0xFFFF;
}
public void setU(int value) {
    regU = value & 0xFFFF;
}
public void setS(int value) {
    regS = value & 0xFFFF;
}
public void setDP(int value) {
    regDP = value & 0xFF;
}

```

```

public void reset() {
    regA = regB = 0;
    regX = regY = regU = regS = 0;
    regDP = 0;
    conditionCodes.clear();
    regPC = memory.readWord(address: 0xFFFFE);
}

```

3.1.5. Interface Utilisateur (optionnel mais recommandé).

- Interface graphique avec Java Swing pour :
 - Charger un programme
 - Afficher les registres en temps réel
 - Step-by-step (exécution pas à pas)
 - Debug (breakpoints).

L'interface ne fait rien elle-même, elle communique avec le cœur du simulateur.

```

package com.motorola6809.view;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.List;
import com.motorola6809.simulator.CPU;
import com.motorola6809.simulator.Memory;
import com.motorola6809.simulator.InstructionAssembler;

public class Simulateur6809 {
    static JMenu menu, submenu, menuSimulation, menuEdition, menuAffichage, menuOutils, menuFenetres;
    static JMenuBar mb;
    static JMenuItem i1, i2, i3, i11;

    static JTable ramTable;
    static JTable romTable;
    static JFrame mainFrame;

    static Memory memory;
    static CPU cpu;

    static JTextField pcField, uField, xField, yField;
    static JTextField aField, bField, dpField, spField;
    static JCheckBox[] flagCheckBoxes;
    static JTextArea instructionArea;
    static JTextArea consoleArea;
    static JTextArea historiqueArea;
    static JTextArea editeur;

    static Timer runTimer;
    static volatile boolean isRunning = false;

```

4. Réalisation (Développement).

Pour réaliser ce projet, nous avons utilisé le langage Java et l'environnement de développement Visual Studio Code (VS Code). Afin de rendre le code plus clair et organisé, nous avons divisé le programme en plusieurs classes.

4.1. Mémoire

Nous avons notamment conçu un module **Mémoire**, avec deux sous-classes : **RAM** et **ROM**, chacune ayant un comportement spécifique pour refléter le fonctionnement réel des mémoires.

Pour gérer les instructions du processeur, nous avons élaboré un **algorithme récapitulatif** du programme, puis analysé chaque composant du processeur

afin de comprendre son rôle. Cette analyse nous a permis de créer des **méthodes adaptées** pour simuler correctement le fonctionnement du processeur.

```
package com.motorola6809.simulator;
public class ROM {
    private static final int MEMORY_SIZE = 65536;
    private byte[] memory;
    private boolean[] initialized;
    public ROM() {
        memory = new byte[MEMORY_SIZE];
        initialized = new boolean[MEMORY_SIZE];
        for (int i = 0; i < MEMORY_SIZE; i++) {
            memory[i] = 0;
            initialized[i] = false;
        }
    }
    public byte readByte(int address) {
        checkAddress(address);
        return memory[address];
    }
    public void writeByte(int address, byte value) {
        checkAddress(address);
        memory[address] = value;
        initialized[address] = true;
    }
    public int readWord(int address) {
        checkAddress(address);
        checkAddress(address + 1);
        int highByte = readByte(address) & 0xFF;
        int lowByte = readByte(address + 1) & 0xFF;
        return (highByte << 8) | lowByte;
    }
    public void writeWord(int address, int value) {
        checkAddress(address);
        checkAddress(address + 1);
    }
    public void load(int address, byte[] data) {
        checkAddress(address);
        if (address + data.length > MEMORY_SIZE) {
            throw new IllegalArgumentException(
                s: "Les données dépasseraient les limites de la mémoire");
        }
        for (int i = 0; i < data.length; i++) {
            memory[address + i] = data[i];
            initialized[address + i] = true;
        }
    }
}
```

```
package com.motorola6809.simulator;
public class RAM {
    private static final int MEMORY_SIZE = 65536;
    private byte[] memory;
    public RAM() {
        memory = new byte[MEMORY_SIZE];
        for (int i = 0; i < MEMORY_SIZE; i++) {
            memory[i] = 0;
        }
    }
    public byte readByte(int address) {
        checkAddress(address);
        synchronized (this) {
            return memory[address];
        }
    }
    public void writeByte(int address, byte value) {
        checkAddress(address);
        synchronized (this) {
            memory[address] = value;
        }
    }
    public int readWord(int address) {
        checkAddress(address);
        checkAddress(address + 1);
        synchronized (this) {
            int highByte = memory[address] & 0xFF;
            int lowByte = memory[address + 1] & 0xFF;
            return (highByte << 8) | lowByte;
        }
    }
    public void writeWord(int address, int value) {
        checkAddress(address);
        checkAddress(address + 1);
        synchronized (this) {
            byte highByte = (byte) ((value >> 8) & 0xFF);
            byte lowByte = (byte) (value & 0xFF);
            memory[address] = highByte;
            memory[address + 1] = lowByte;
        }
    }
    public void load(int address, byte[] data) {
        checkAddress(address);
        if (address + data.length > MEMORY_SIZE) {
            throw new IllegalArgumentException(

```

```

package com.motorola6809.simulator;

public class Memory {
    private static final int MEMORY_SIZE = 65536;
    private RAM ram;
    private ROM rom;
    private boolean[] romMap;
    public Memory() {
        ram = new RAM();
        rom = new ROM();
        romMap = new boolean[MEMORY_SIZE];
        for (int i = 0; i < MEMORY_SIZE; i++) {
            romMap[i] = false;
        }
    }
    private void checkAddress(int address) {
        if (address < 0 || address >= MEMORY_SIZE) {
            throw new IllegalArgumentException(
                String.format("Adresse 0x%04X hors limites (0x0000-0xFFFF)", address));
        }
    }

    public byte readByte(int address) {
        checkAddress(address);
        if (romMap[address]) {
            return rom.readByte(address);
        } else {
            return ram.readByte(address);
        }
    }
    public void writeByte(int address, byte value) {
        checkAddress(address);
        if (!romMap[address]) {
            ram.writeByte(address, value);
        } else {
            rom.writeByte(address, value);
        }
    }
    public int readWord(int address) {
        checkAddress(address);
        checkAddress(address + 1);
        int highByte = readByte(address) & 0xFF;
        int lowByte = readByte(address + 1) & 0xFF;
        return (highByte << 8) | lowByte;
    }
    public void writeWord(int address, int value) {

```

4.2. CPU

Le module CPU gère également les instructions du processeur et leurs différents modes d'adressage (immédiat, direct, étendu, etc.). Chaque instruction est décodée pour déterminer le mode utilisé, puis exécutée en conséquence, ce qui permet au simulateur de reproduire fidèlement le fonctionnement du Motorola 6809. Cette gestion assure que les opérations sur les registres et la mémoire respectent le comportement réel du processeur.

```

package com.motorola6809.simulator;
import java.util.EnumSet;
public class CPU {
    private final Memory memory;
    private int regA;
    private int regB;
    private int regX;
    private int regY;
    private int regU;
    private int regS;
    private int regPC;
    private int regDP;
    private EnumSet<Flag> conditionCodes = EnumSet.noneOf(elementType: Flag.class);
    public CPU(Memory memory) {
        this.memory = memory;
    }

    public void initialize() {
        reset();
    }
    public int getRegA() {
        return regA;
    }
    public int getRegB() {
        return regB;
    }
    public int getRegPC() {
        return regPC;
    }
    public void reset() {
        regA = regB = 0;
        regX = regY = regU = regS = 0;
        regDP = 0;
        conditionCodes.clear();
        regPC = memory.readWord(address: 0xFFFF);
    }
    public void step() {
        int opcode = fetchByte();
        switch (opcode) {
            case 0x86:
                opLDAimmediate();
                break;
            case 0xC6:
                opLDBimmediate();
                break;

```

Dans le simulateur, chaque instruction du processeur est implémentée comme une méthode spécifique dans le code. Ces méthodes prennent en compte le mode d'adressage utilisé (immédiat, direct, indexé, étendu, etc.) et effectuent les opérations correspondantes sur les registres et la mémoire. Cette approche modulaire permet de gérer chaque instruction séparément, de simplifier le débogage et de garantir que le simulateur reproduise fidèlement le comportement réel du Motorola 6809.

```
    }  
    return effectiveAddress;  
}  
private void opLDAimmediate() {  
    regA = fetchByte();  
    updateNZFlags(regA);  
    clearVFlag();  
}  
private void opLDBimmediate() {  
    regB = fetchByte();  
    updateNZFlags(regB);  
    clearVFlag();  
}  
private void opLDXimmediate() {  
    regX = fetchWord();  
    updateNZFlags16(regX);  
    clearVFlag();  
}  
private void opLDYimmediate() {  
    regY = fetchWord();  
    updateNZFlags16(regY);  
    clearVFlag();  
}  
private void opLDUimmediate() {  
    regU = fetchWord();  
    updateNZFlags16(regU);  
    clearVFlag();  
}  
private void opLDSimmediate() {  
    regS = fetchWord();  
    updateNZFlags16(regS);  
    clearVFlag();  
}  
private void opLDDimmediate() {  
    int value = fetchWord();  
    setD(value);  
    updateNZFlags16(value);  
    clearVFlag();  
}  
private void opSTAdirect() {  
    int offset = fetchByte() & 0xFF;  
    int addr = (regDP << 8) | offset;  
    memory.writeByte(addr, (byte) regA);  
}
```

4.3. Interface Graphique.

Le simulateur dispose d'une **interface graphique simple et claire**, conçue pour afficher les **registres**, la **mémoire** et l'état du **processeur** en temps réel. Elle permet également d'exécuter le programme **pas à pas** ou en **mode continu**, de charger des instructions et de visualiser facilement les résultats, facilitant ainsi le suivi et le débogage du fonctionnement du Motorola 6809.

```
package com.motorola6809.view;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.List;
import com.motorola6809.simulator.CPU;
import com.motorola6809.simulator.Memory;
import com.motorola6809.simulator.InstructionAssembler;

public class Simulateur6809 {
    static JMenu menu, submenu, menuSimulation, menuEdition, menuAffichage, menuOutils, menuFenetres;
    static JMenuBar mb;
    static JMenuItem i1, i2, i3, i11;

    static JTable ramTable;
    static JTable romTable;
    static JFrame mainFrame;

    static Memory memory;
    static CPU cpu;

    static JTextField pcField, uField, xField, yField;
    static JTextField aField, bField, dpField, spField;
    static JCheckBox[] flagCheckBoxes;
    static JTextArea instructionArea;
    static JTextArea consoleArea;
    static JTextArea historiqueArea;
    static JTextArea editeur;

    static Timer runTimer;
    static volatile boolean isRunning = false;

    Run | Debug
    public static void main(String[] args) {
        memory = new Memory();
        cpu = new CPU(memory);
        cpu.initialize();
    }
}
```

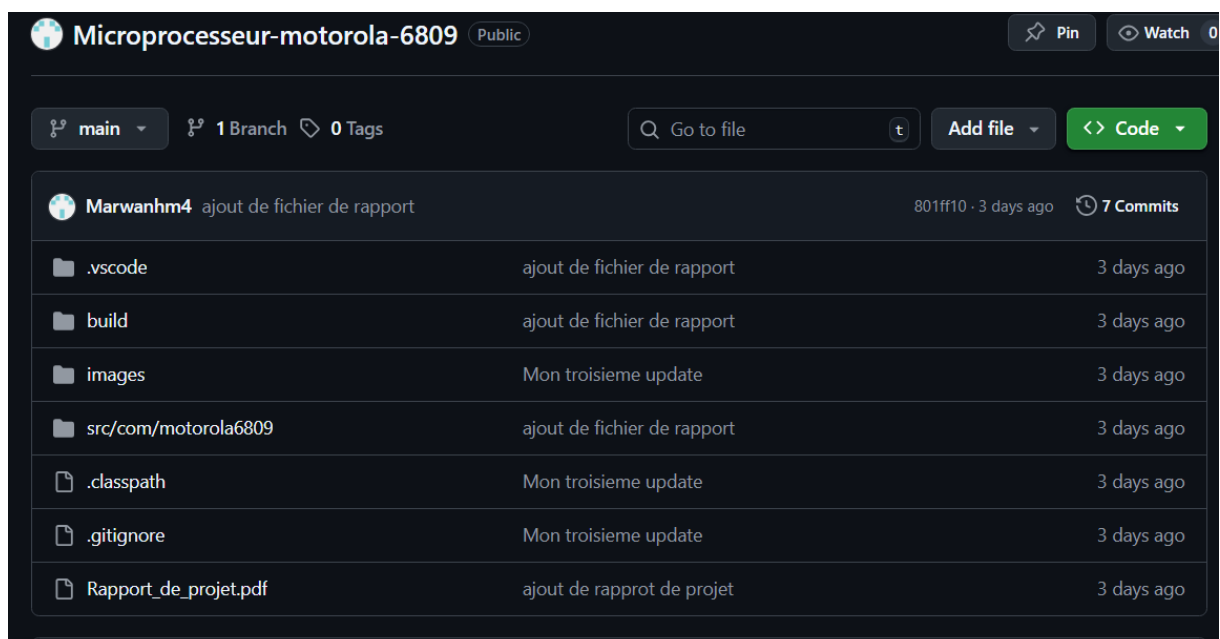
L'interface graphique simple affiche les **registres**, la **mémoire** et l'état du CPU en temps réel. Elle communique directement avec le **CPU** pour exécuter les instructions pas à pas ou en continu.

Ainsi, toutes les actions de l'interface reflètent fidèlement le fonctionnement interne du Motorola 6809.

5.Travail en groupe.

Au début, nous avons rencontré de nombreuses difficultés concernant la **synchronisation** entre nous. Il était compliqué de gérer deux codes développés par deux personnes différentes dans un seul projet. Pour y remédier, nous avons décidé de **diviser les tâches** : l'un se chargeait du développement de l'**interface graphique**, tandis que l'autre s'occupait du **backend**. Tous les deux avons ensuite travaillé sur la **connexion entre les deux parties**.

Par la suite, nous avons utilisé **Git et GitHub**, ce qui a grandement facilité l'organisation du projet. Ces outils nous ont permis de **travailler dans le même dossier** tout en **synchronisant notre travail grâce aux commits**, rendant la collaboration beaucoup plus efficace.



Pour bien travailler avec Git et GitHub, nous avons utilisé plusieurs commandes essentielles afin de synchroniser correctement les tâches développées par chaque membre de l'équipe. Ces commandes nous ont permis d'ajouter les modifications, de créer des commits, d'envoyer notre travail vers le dépôt en ligne et de récupérer les mises à jour des autres. Grâce à cela, nous avons assuré une bonne organisation du projet, une coordination fluide entre les développeurs et une gestion efficace des versions du code.

Versionner en local

git init	initialise le dépôt (se mettre sur le bon dossier), mieux à faire depuis Github.com
git add .	ajoute toutes les modifications (le . symbolise tout)
git commit -m "explication"	créer un nouveau commit. git add pousse les fichiers en zone d'index, git commit les sauvegarde réellement dans un nouveau commit

Versionner sur un dépôt distant

git clone lien-github.com	récupérer travail depuis dépôt distant
git push -u origin master	pousse les modifications vers serveur
git push -f origin master	pousse de force des modifications (à manipuler avec précaution)

Commit son projet sur Github

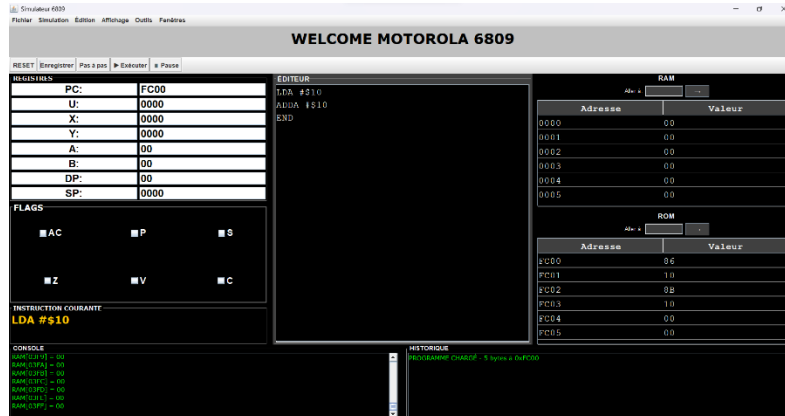
```
git add .  
git commit -m "message"  
git push -u origin master
```

6. Résultats & Tests.

Au début du projet, nous avons commencé par tester les instructions et quelques commandes directement dans le terminal, avant d'établir la connexion entre le backend et l'interface graphique. Une fois l'intégration réalisée, nous avons procédé à des tests complets du programme afin d'identifier ses limites, d'effectuer les corrections

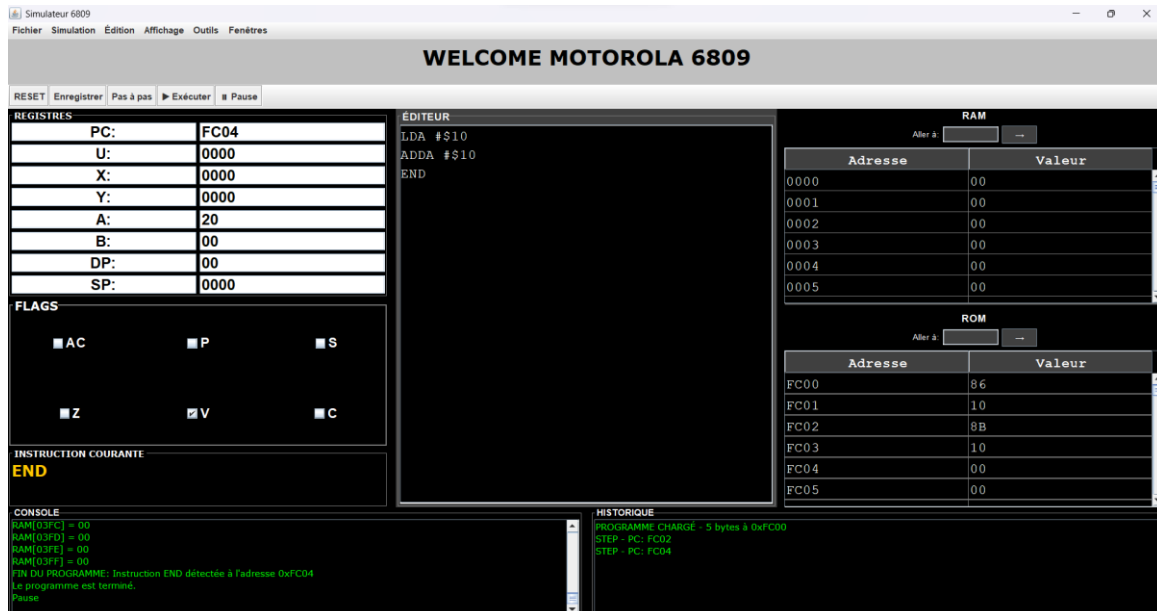
nécessaires et d'apporter des améliorations pour optimiser son fonctionnement.

Voici des exemples avec notre interface :



Après l'exécution d'un simple programme :

```
LDA #$10
ADDA #$10
END
```



Le processeur commence l'exécution à l'adresse **FC00** et charge l'instruction LDA #\$10, ce qui place la valeur 10h dans le registre **A**.

Ensuite, il passe à l'adresse **FC02** pour exécuter ADDA #\$10, ce qui ajoute 10h à A et donne comme résultat 20h.

Le compteur ordinal (PC) avance ensuite à l'adresse **FC04**, où il détecte l'instruction END qui arrête l'exécution du programme.

À la fin, le registre **A** contient **20h**, le flag **V** est activé, et le message de fin d'exécution apparaît dans la console et l'historique.

7. Conclusion et Remerciements .

Ce projet nous a offert une véritable expérience de travail en équipe et de gestion de projet du début à la fin. Il nous a permis d'apprendre à organiser les responsabilités, à synchroniser nos tâches, à utiliser les bons outils de collaboration et à maintenir une cohérence entre l'interface et le code. Malgré les difficultés rencontrées et le stress lié à notre première expérience de développement en groupe, nous avons appris à nous adapter, à communiquer efficacement et à gérer la pression. Au final, ce projet représente une étape importante dans notre apprentissage et nous sommes fiers du travail accompli ainsi que des résultats obtenus.

Un grand merci à notre professeur Hicham Benaalla , qui nous motive chaque jour à donner la meilleure version de nous-mêmes. Grâce à son accompagnement, nous réalisons des projets concrets qui renforcent nos compétences et enrichissent nos carrières. Il nous prépare avec sérieux et détermination à intégrer le marché du travail avec confiance et professionnalisme.